

김희경 — 포트폴리오

데이터 플랫폼 엔지니어

☎ 010-9601-2589 ✉ hshsnamu@gmail.com 🌐 github.com/gimgit 📅 Updated 2026-08-01

본 문서는 이력서에 있는 주요 프로젝트에 대해 왜 필요했는지와 설계 의사결정 배경을 중심으로 서술합니다 — AI 캐릭터 챗 서비스의 LLM 대화 로그 198개 테이블·약 130억 행을 Iceberg 레이크하우스로 옮기며 레이아웃·프루닝·비용 구조를 다시 설계한 과정(①), 단일 프로젝트 BigQuery를 13개 프로젝트 거버넌스 체계로 재설계한 과정(②)과 그 접근 통제를 실제로 동작하게 만든 컬럼 보안등급 자동화(③), 그리고 재사용 가능한 IaC를 검증했던 AWS 마이그레이션 PoC(④) 순입니다.

1. BigQuery → Apache Iceberg Lakehouse 마이그레이션 | 뮌헨 테크놀로지스

06/2026 - 현재

작업의 필요성

- 핵심 데이터들이 BigQuery 단일 스택에 묶여 **유량 증가에 따라 데이터 처리 비용이 늘어나는 구조**를 해결하고, ISMS-P 심사에 대비해 폐쇄망에서 **데이터 파이프라인 재구축**이 필요했습니다. 대상은 198개 테이블 · 약 130억 행 · 8.2TB(2026-08 Iceberg 스냅샷 실측)이며, CDC가 흐르는 운영 파이프라인 위에서 옮겨야 했습니다

아키텍처 전환의 핵심 — 관리형 CDC apply를 직접 책임하는 구조로

- BigQuery에서는 CDC Connector가 UPSERT·DELETE 이벤트를 테이블에 스트리밍하면 BigQuery 내부기능인 background apply를 통해 변경분을 실제 테이블 상태에 반영했습니다.
- S3+Iceberg로 이전한 뒤에는 이 관리형 계층이 사라졌습니다. 단순히 저장 위치만 옮기는 것이 아니라, CDC 변경분을 주기적으로 테이블에 반영하고 그 과정에서 발생하는 **파일 재작성, delete 파일 누적, compaction 및 읽기 비용**을 데이터 플랫폼에서 직접 최적화해야 했습니다.
- BigQuery가 내부적으로 수행하던 CDC merge를 Iceberg 환경에서 직접 구현했습니다. 초기 대량 적재의 OOM은 데이터 분배 방식을 바꿔 해결하고, 운영 단계에서는 CDC 변경량과 조회 특성에 따라 쓰기방식을 최적화 했습니다. 이후 파일 간 키 범위 중첩과 Parquet row-group 크기까지 조정해 쓰기 비용과 조회 성능을 함께 관리했습니다.

읽기 최적화 — Parquet 파일 열기 최소화 + row-group 읽기 최소화

- 프루닝 지점을 두 층으로 나눠 층별 수단을 분리했습니다
- [층 1] Iceberg 메타데이터(파일 min-max): 파일을 아예 열지 않게
- [층 2] Parquet footer(row-group): 파일에서 덜 읽게

층 1. 파일이 얼마나 겹쳐 있는지 관측하고 overlap depth를 1에 수렴하도록 관리

- 현상** : 특정 값을 조회할 때 조건에 맞지 않는 파일까지 대량으로 열렸습니다. Iceberg는 파일마다 키 범위(min-max)를 기록해 범위 밖 파일을 열지 않고 건너뛰는데, **파일들의 범위가 서로 겹치면 이 장치가 무력해집니다**

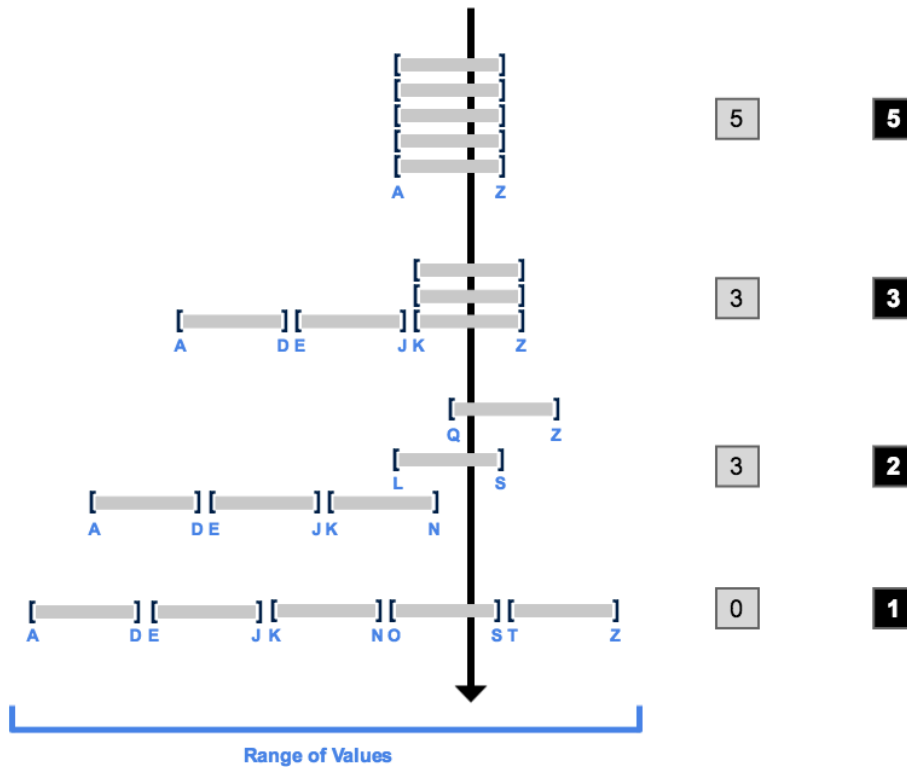
파일 범위 (예시 키 A ~ Z)	키 B 조회 시 여는 파일
A~C · D~F · G~I	1개
A~Z · A~Y · A~X	3개 전부 — 읽는 양 3배

- 원인** : Iceberg(1.5.2)에는 파일 간 키 범위 겹침을 정량화해 재작성 대상을 선별하는 내장 지표가 없습니다. Iceberg 최적화의 기준은 파일 크기가 유일했고 "크기는 정상이지만 키 범위만 넓게 겹친 파일"은 애초에 정리 대상으로 잡히지 않습니다.
- 조치** : snowflake의 overlap depth 개념을 이식해 **depth_observe**를 구현했습니다. 키 범위가 겹치는 파일 수를 세는 지표로, 파일별 min-max가 이미 메타데이터에 기록돼 있어 data file을 직접 읽지 않고도 정리가 필요한 파일을 관측하고 **sort rewrite로 overlap depth를 1에 수렴하도록 최적화**했습니다.

Micro-partitions (Total) = 5

Overlapping
Micro-partitions

Overlap
Depth



Snowflake의 overlap depth: 구간이 겹칠수록 한 키 조회가 여는 파티션(depth)이 커지고, well-clustered 상태는 depth 1로 수렴 (그림: Snowflake Docs). 이 지표를 depth_observe로 이식·계측하고 sort rewrite로 depth 1 수렴을 유지

층 2. MoR 테이블 row-group 크기 조정 — 128MB → 16MiB

- **현상** : 무작위 키로 특정 행을 조회하는 패턴에서 스캔량이 과도했습니다. 필요한 건 한 행인데 Parquet row-group 128MB 덩어리 단위로만 건너뛸 수 있으면, 한 행을 읽기 위해 오버헤드를 포함해서 128MB를 통째로 읽습니다.
- **조치·결과** : row-group을 16MiB로 조정해 Parquet 내부에서도 읽는 데이터를 줄였습니다. **대형 테이블 ETL과정 스캔 바이트 6TB → 약 0.5TB (92% 감소)**
- **기본값 방향의 판단** — 축소는 압축률 하락·요청 수 증가의 대가가 따르므로, **실패 비용의 비대칭**을 기준으로 삼았습니다

	불필요하게 적용했을 때	필요한데 적용 안 했을 때
손실	압축률 % 단위	읽기량 수 배
되돌리기	설정 변경 후 재작성분부터 반영 (가역)	테이블 전량 재작성 (비가역)

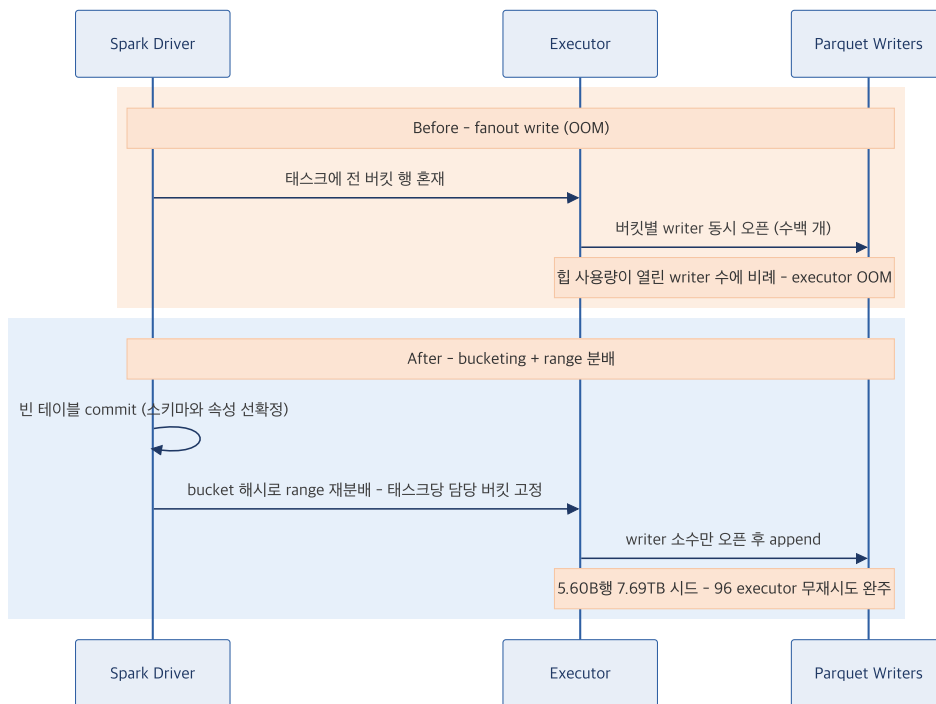
- 한쪽은 작고 가역, 다른 쪽은 크고 비가역 — **되돌리기 쉬운 쪽을 기본값으로** 두는 것이 합리적이라 판단해 축소를 기본 방향으로 채택했습니다

쓰기 최적화 — OOM 없는 대량 적재 + write amplification 통제

- 쓰기 경로의 문제를 두 축으로 나눠 접근했습니다 — 대량 Upsert시 **OOM 없도록** / CDC 변경 반영의 재작성 비용(write amplification)을 테이블 특성에 맞게

축 1. 대량 Upsert OOM 해소 — 빈 테이블 선택 및 + bucketing

- **현상** : 한 배치에 넓은 키 범위를 Upsert하는 정기 배치에서 executor **OOM**이 반복됐습니다. 데이터 크기가 아니라 **쓰는 쪽 메모리**가 문제였습니다
- **원인** : fanout write 구조. Upsert 대상 키 범위가 넓으면 **버킷 수만큼 Parquet writer를 동시에 열고**, 힙 사용량이 열린 writer 수에 비례해 커져 scale out 또는 scale in으로도 한계가 있었습니다.
- **조치** : 쓰기를 2단으로 분리했습니다. ① **빈 테이블 선택 및** — 스키마·bucket spec·테이블 속성(레이아웃 계약)만 먼저 확정. 분배 기준이 카탈로그에 존재해야 다음 단계가 가능합니다 ② **bucket 해시 range 재분배 후 append** — 태스크당 담당 버킷을 고정해 writer를 소수만 열게 함
- **결과** : prod 최대 테이블 **5.60B행 · 7.69TB**를 **2h17m · 96 executor · 무재시도**로 시드 완료. 처음엔 버킷을 읽기 프루닝 수단으로 이해하고 도입했지만, 이 경험으로 버킷의 1차 가치는 읽기 프루닝이 아니라 **커밋 단위 분할과 쓰기 병렬성**이라고 판단을 고치게 됐습니다



대량 쓰기 OOM의 원인과 해소 — fanout write는 열린 writer 수에 힙이 비례해 OOM, 빈 테이블 선택밋으로 레이아웃 계약을 확정한 뒤 bucket 해시 + range 분배로 태스크당 담당 버킷을 고정해 무재시도 완주

축 2. Copy-on-Write / Merge-on-Read 판정 룰 — write/read amplification 균형의 코드화

- **문제** : CoW는 한 행만 바뀌어도 해당 파일 전체를 다시 쓰고(write amplification), MoR은 변경분만 delete 파일로 기록하는 대신 읽을 때 병합 비용을 냅니다(read amplification). 어느 쪽이 유리한지는 테이블의 크기·변경 패턴의 함수인데, 모드 선택이 **일회성 판단으로 이뤄져 오분류가 남는 구조**였습니다
- **조치** : 아래와 같은 판정 프레임워크로 쓰기모드를 관리하기 시작했습니다.

순서	조건 (위에서부터 first-match)	판정
R0	사실상 append-only (변경 비율 < 1%)	CoW
R1	크기 ≥ 1TB (풀 재작성 절대비용)	MoR
R2	변경 ≥ 10,000행/h AND ≥ 10GB	MoR
R3	나머지	CoW

- **결과** — 룰을 기존 테이블에 돌리자 **이전 선정에 크기 축이 통째로 빠져 있었다**는 결함이 드러났습니다. 3종을 CoW로 강등하고 630GB급 포함 4종을 MoR로 신규 편입 — 판정 프레임워크가 실제로 오류를 잡아낸 사례입니다

기여 — [리드] 레이아웃·쓰기모드 판정 룰 설계, compaction·프루닝 전략 수립, 정합성 검증 체계 및 관측 도구 개발, 장애 RCA 주도 (대규모 백필·인프라 구축 팀 분담)

2. BigQuery 마이그레이션 및 거버넌스 도입 | 뽀튼 테크놀로지스

02/2026 - 현재

작업의 필요성

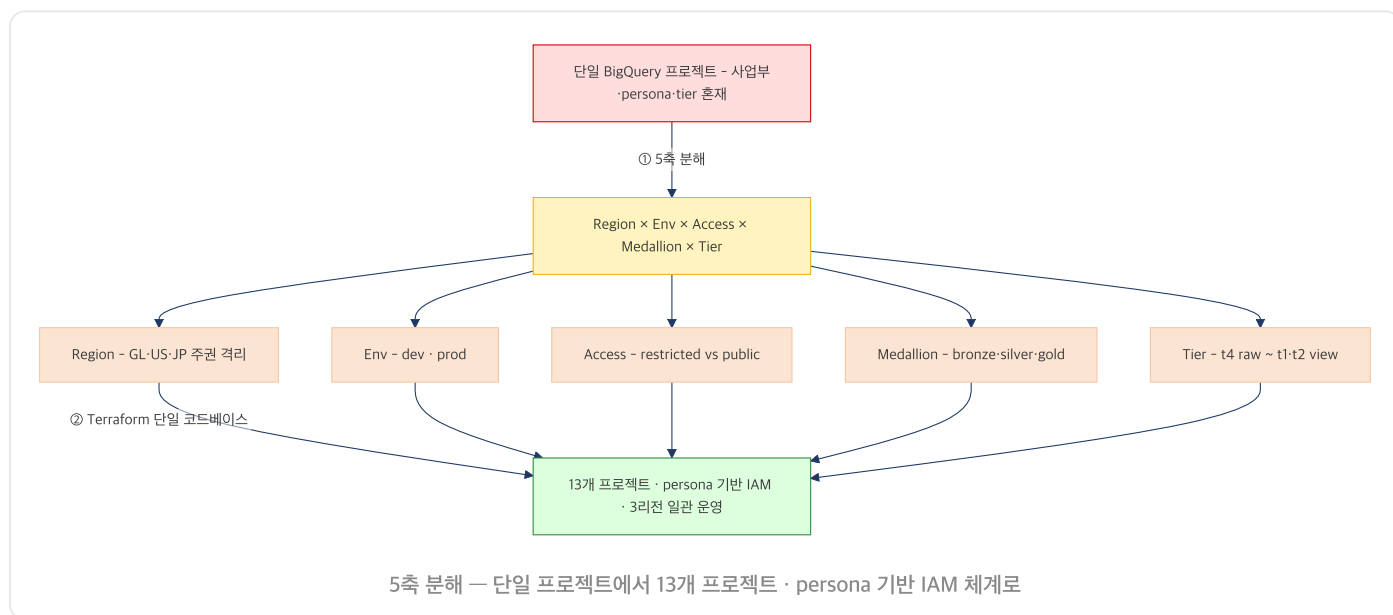
- 단일 BigQuery 프로젝트에 전 사업부·persona·보안등급의 데이터가 혼재 — 리전별 **데이터 주권 격리도**, **raw 접근 통제도** 프로젝트 경계로는 표현할 수 없는 구조였습니다. **이관 규모는 레거시 기준 테이블 1,600+ · 500TB+** — 운영 중인 파이프라인 위에서 옮겨야 했습니다.

설계 배경과 판단

- **5축(Region × Env × Access × Medallion × Tier)**으로 분해해 13개 프로젝트로 재설계 — 프로젝트 경계로 승격한 축은 Region·Env·Access 셋뿐(12 + 분석 전용 1), Medallion·Tier는 dataset 네이밍으로 흡수해 조합 폭발(~144개) 없이 정책이 경계 자체에 표현되도록
- **raw 권한 0 default** — 사람 계정은 restricted 프로젝트에 직접 권한이 없고 authorized view를 통해서만 소비. 권한 변경의 영향 범위가 view 단위로 격리됨
- **persona 기반 RBAC** — IAM을 전면 리팩터링, tier별 role cap·사용자/서비스 그룹 분리로 개인 단위 권한 부여를 제거
- **laC** — 테라폼을 단일 진실 공급원으로 삼아 CI/CD로 BigQuery 자원을 관리하는 감사 가능한 인프라 운영
- **N-리전 DRY** — 리전 추가가 tfvars PR + values.yaml PR 2건으로 끝나도록 Terraform 모듈화, GitHub Actions matrix가 리전별 병렬 auto-apply

제약·트레이드오프

- 운영 중인 파이프라인 위에서의 이관이라 **이관-운영 병행 정합성 관리**가 상수 제약
- 적재 신선도는 비용과의 트레이드오프 — 마이그레이션 과정에서 CDC 적재 주기(max_staleness)를 표준화해 background apply 부하를 구조적으로 절감 (정량 수치는 이력서 참조)
- **프로젝트 분리의 대가** — 크로스 프로젝트 참조와 프로젝트별 reservation 관리 부담이 생기지만, 상시 authorized view 경로와 Terraform 모듈 일원화로 흡수



기여 — [리드] 5축 분해·persona IAM 모델 설계, Terraform 모듈 구현, 멀티 리전(GL/US/JP) 운영

3. BigQuery 컬럼 보안 등급 자동화 시스템 (HITL) | 휘튼 테크놀로지스

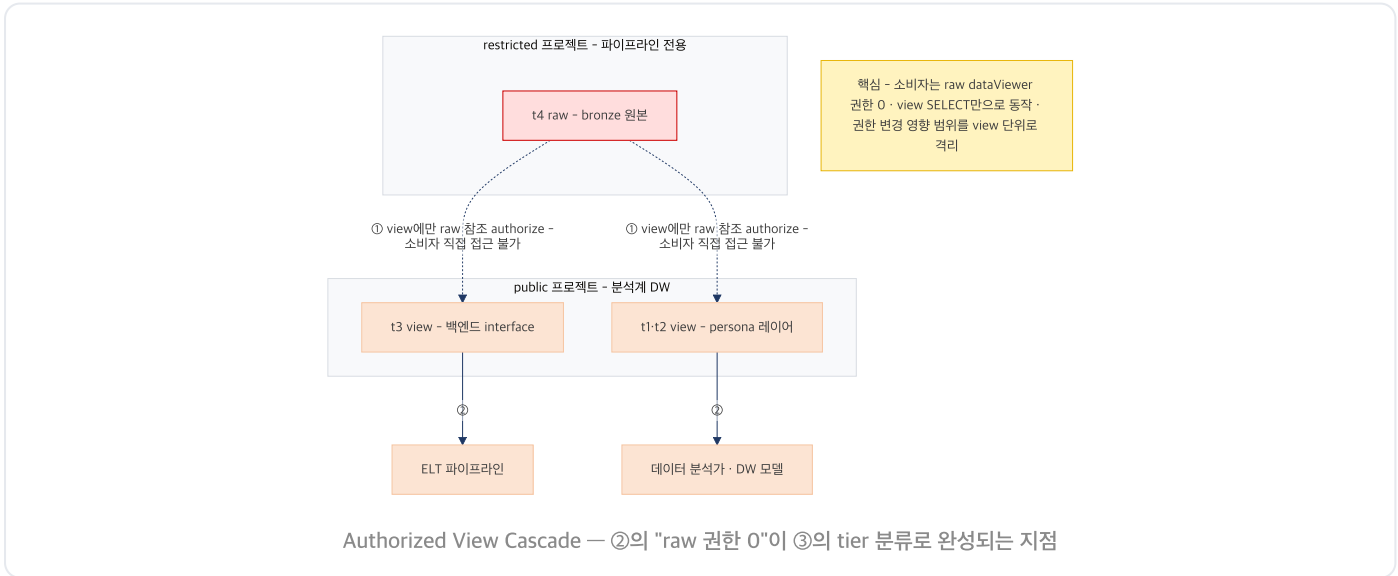
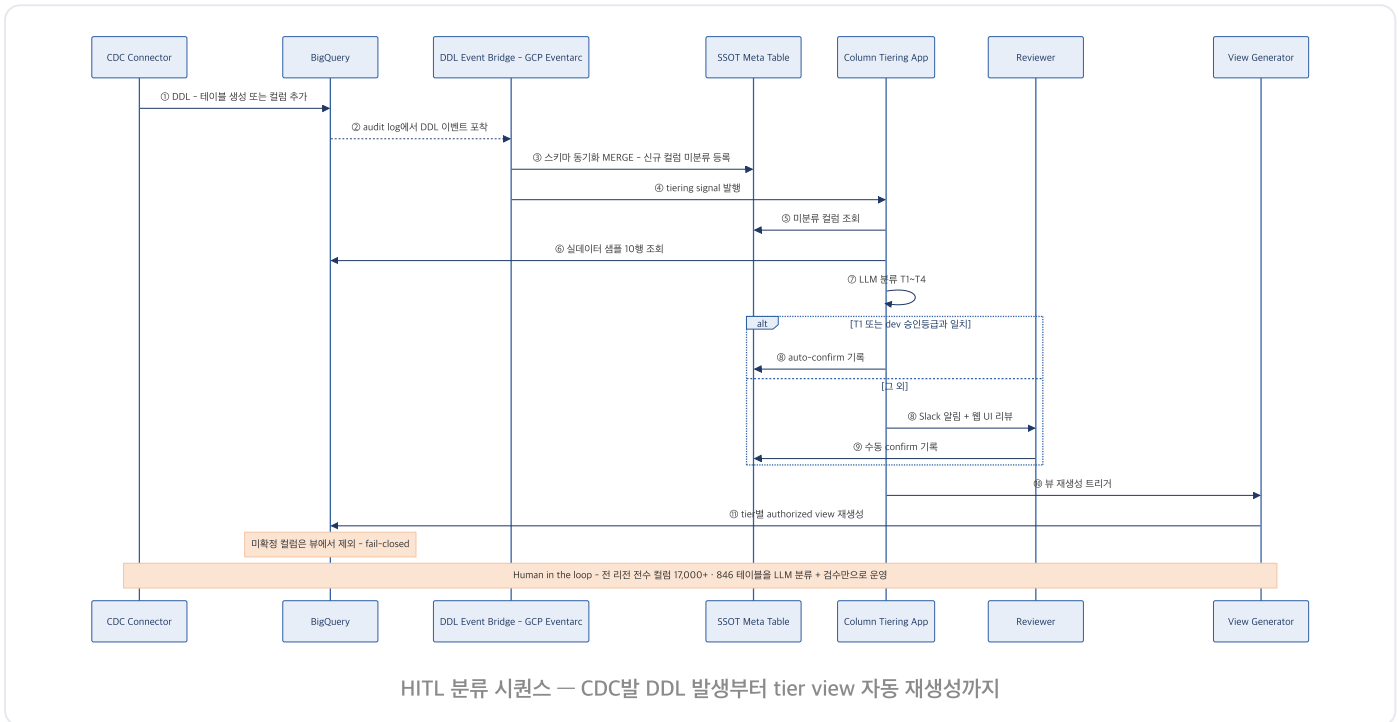
02/2026 - 04/2026

작업의 필요성 — ②의 접근 통제를 동작시키는 전제 조건

- ②에서 설계한 raw 접근 차단단의 실행 단위는 **tier별 authorized view** — raw 접근은 view에 위임되고, 소비자 권한은 persona IAM이 view dataset 단위로 바인딩됩니다. 이 체계가 동작하려면 **모든 컬럼에 보안등급(tier)이 먼저 부여되어** 있어야 하는데, **전 리전 분류 대상 전수 기준 컬럼 17,000+ · 846테이블**은 수작업 분류로 감당할 수 없는 규모였습니다.

설계 배경과 판단

- LLM 자동 분류를 도입하되 **확정 권한은 사람에게** — Human-in-the-loop로, 오분류가 곧 접근 통제 오류로 직결되는 리스크를 차단
- **column_security_policy 메타 테이블을 tier의 SSOT로 설계** — 분류 결과와 관리 책임이 한 곳에 기록되고, tier별 authorized view는 여기서 자동 파생(DRY). policy tag 대신 view 파생을 택해 3리전 13프로젝트에 taxonomy 복제 없이 Terraform과 통합, 원본 스키마는 불가침
- **리뷰 병목은 기술이 아니라 정책(SLA)으로 해소** — 분류 정확도를 올려도 사람 검수 처리량이 따라오지 못해 tier 확정이 적체됐습니다. 리뷰어는 전수 검수를 원칙으로 했고, 저는 적체가 곧 fail-closed 구조의 가용성 저하로 이어진다고 봤기에 리스크가 낮은 하위 tier의 검수 기한(SLA)을 분리하고 일괄 승인 경로를 두는 이원화 정책을 제안해 리뷰어와 합의했습니다 — 기술로 해결되지 않는 병목을 협상으로 푼 사례입니다
- 신규 컬럼은 감사 로그 Pub/Sub로 감지되어 분류 큐에 유입 — tier 확정 전 컬럼은 어떤 tier view에도 포함되지 않는 **fail-closed**라 분류 누락이 노출로 이어지지 않는 구조



기여 — [리드] 거버넌스 설계, FastAPI 앱 개발, CI/CD·K8s 배포 전 과정 수행

4. 데이터 플랫폼 현대화 PoC | 컨센트릭스

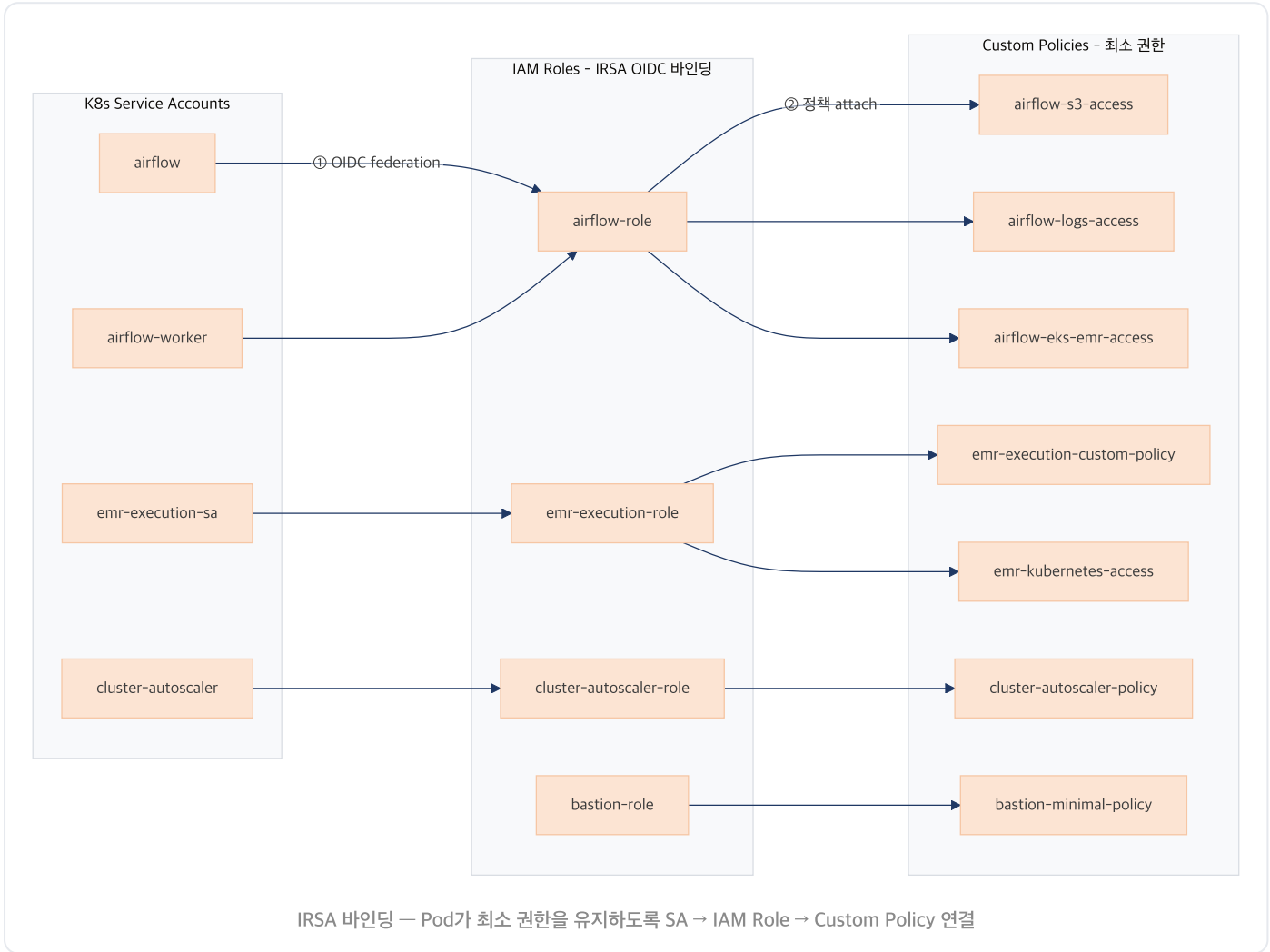
08/2025 - 11/2025

작업의 필요성

- 고객사 온프레미스 YARN·Spark 파이프라인의 AWS 이전 타당성 검증 — 고객사가 늘 때마다 반복될 **환경 구축을 재사용 가능한 IaC로 단축할 수 있는**지가 핵심 질문이었습니다.

설계 배경과 판단

- 배치가 하루 9시간에 몰리는 워크로드 특성상 상시 클러스터 대신 **EMR on EKS + Spot** — Cluster Autoscaler가 배치 시간에만 노드를 확장
- Pod 권한은 **IRSA로 최소화** — Service Account → IAM Role → Custom Policy 단위 바인딩
- 프라이빗 EKS + 바스천 호스트 + GitLab Runner 운영 모델로 외부 노출 차단
- Terraform 모듈화의 검증 기준을 "재현성"으로 — **70여 개 리소스 destroy → apply 재배포**로 실증



비용 비교 — 온프레미스 대비 최대 46% 절감

- 가정: ap-northeast-2, 12 vCPU / 48GiB 노드 6대, 배치 9시간/일 × 30일 = 270시간/월, Airflow 상시 1대 별도
- 할인 전제(공정성 명시): 온프레미스 가정 = AWS RI 3년 약정 50% 할인, Spot = 50% 할인

시나리오	월 비용	비고
기존 YARN 클러스터	₩3,720,000	
EMR on EKS + Spot	₩1,990,000	약 46% 이상 절감